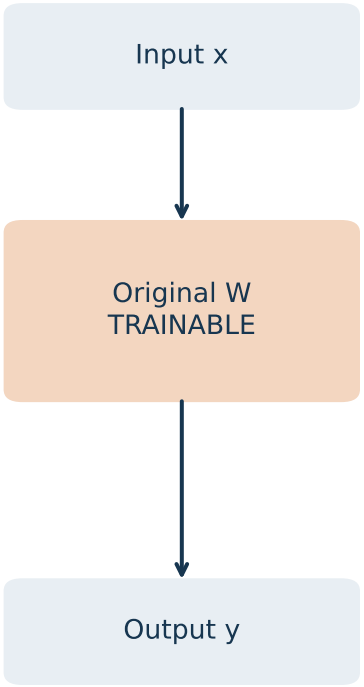
How LoRA changes training

Same pretrained starting weights and the same new training data

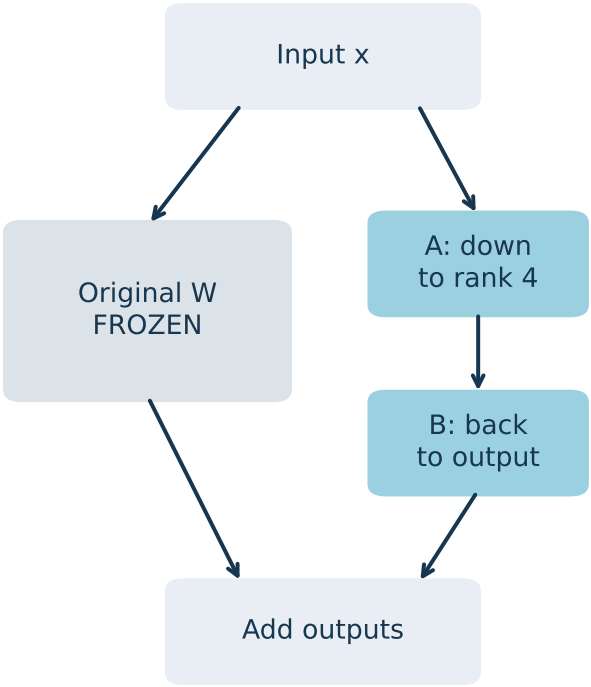
Both approaches learn from our basketball examples. Full fine-tuning lets every original model parameter change. LoRA freezes the original parameters and learns small added matrices. The diagram shows one adapted linear projection; the experiment applies adapters throughout the transformer.

FULL FINE-TUNING



y = Wx

LORA FINE-TUNING



y = Wx + (alpha / rank) BAx

Only the blue A and B matrices train in the LoRA model. Rank = 4; alpha = 8. Biases are omitted from this conceptual diagram. LoRA adapters were applied to QKV, attention-output, and both MLP projections in all 12 transformer blocks.

163,037,184

trainable in full fine-tuning

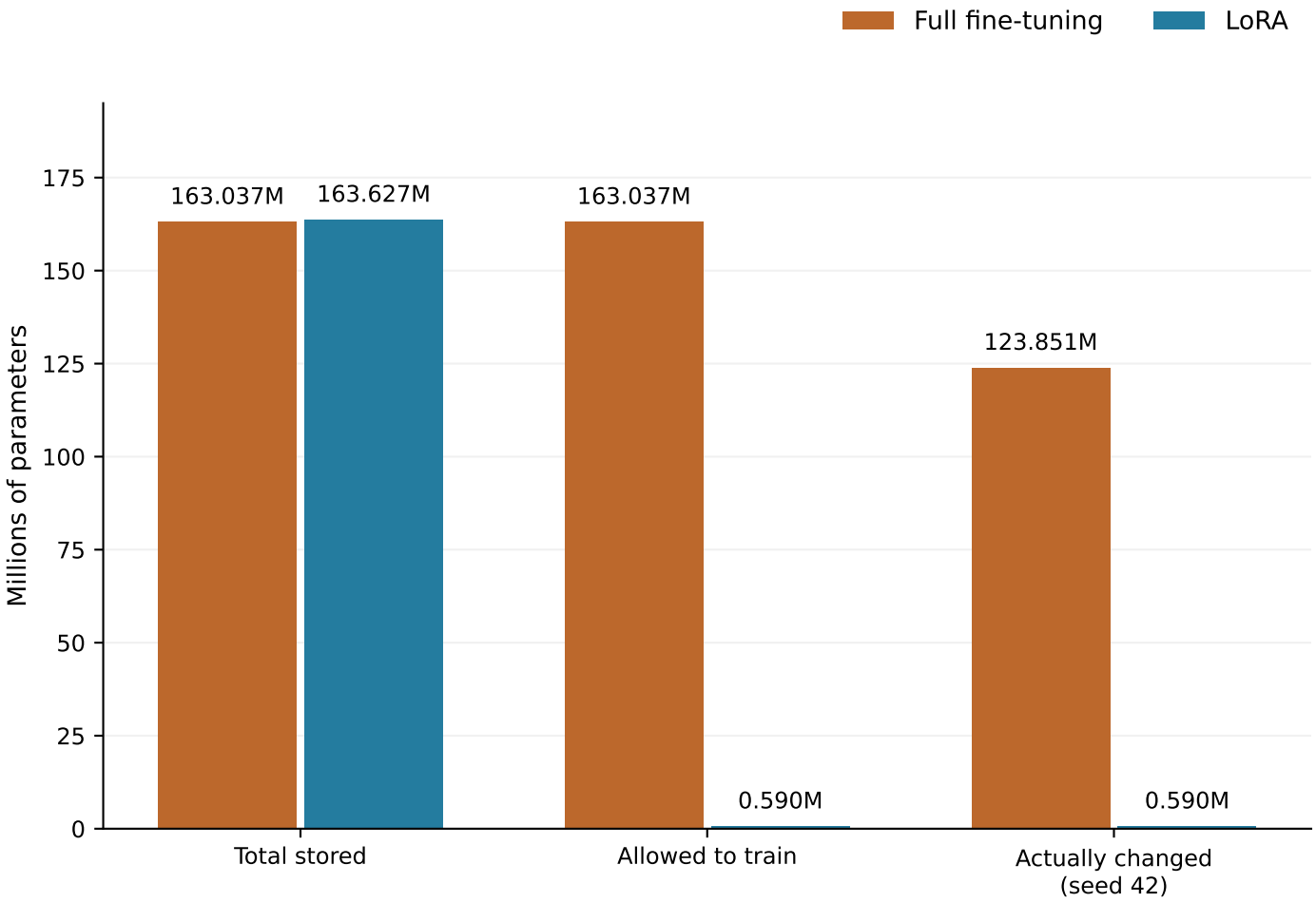
589,824

trainable with LoRA

The untouched pretrained model is a third reference: it receives no task training. The two trained models use identical examples, sample order, batch size, and step count. This isolates parameter-efficient adaptation rather than comparing training against no training.

Total, trainable, and actually changed

These are different quantities; all counts refer to individual scalar values



Total: every model parameter, including frozen weights. Trainable: parameters allowed to receive optimizer updates. Actually changed: individual values differing from their pre-training snapshots at the selected checkpoint. Being trainable does not guarantee a value changed.

276.4x fewer

trainable parameters with LoRA

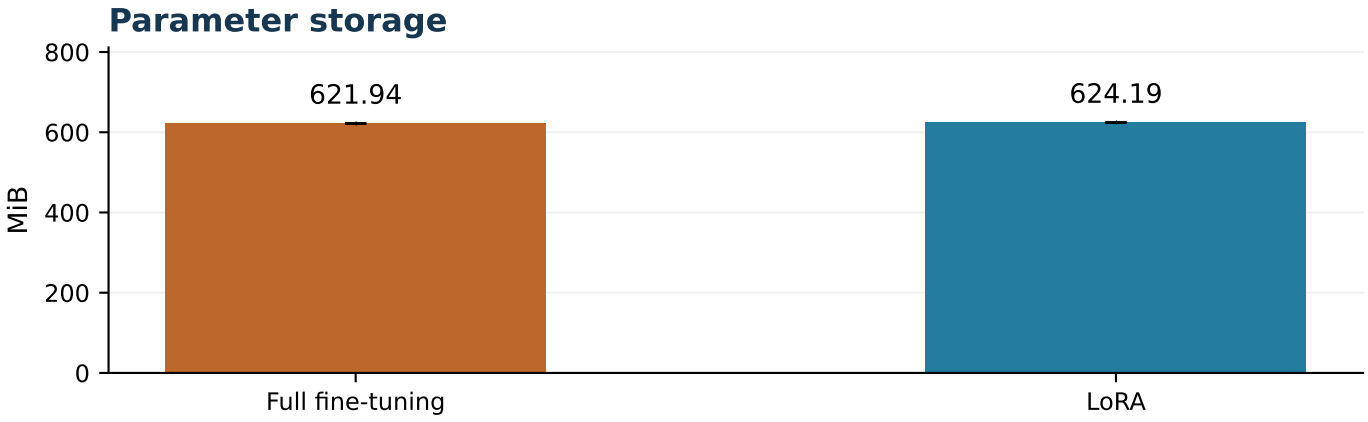
0

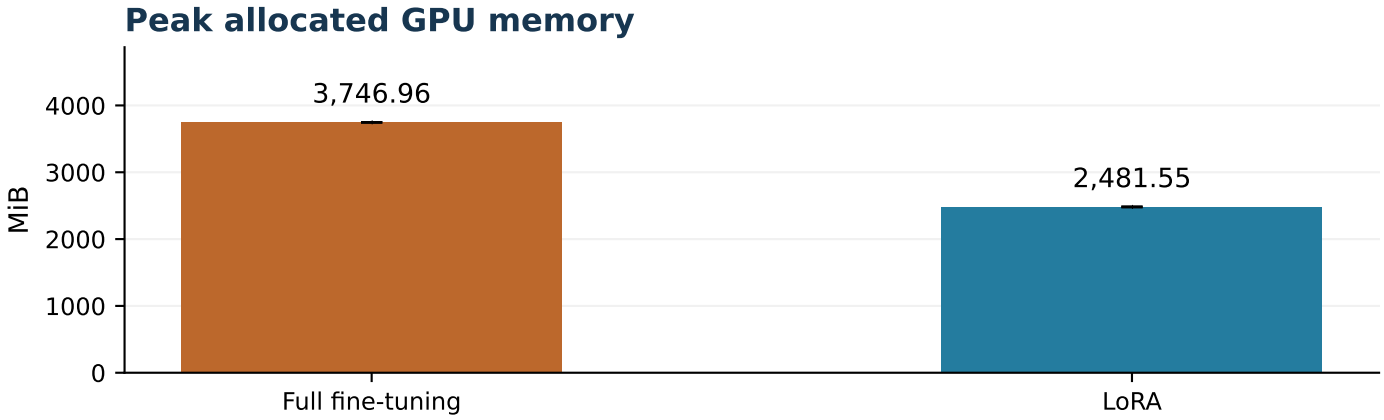
frozen base values changed

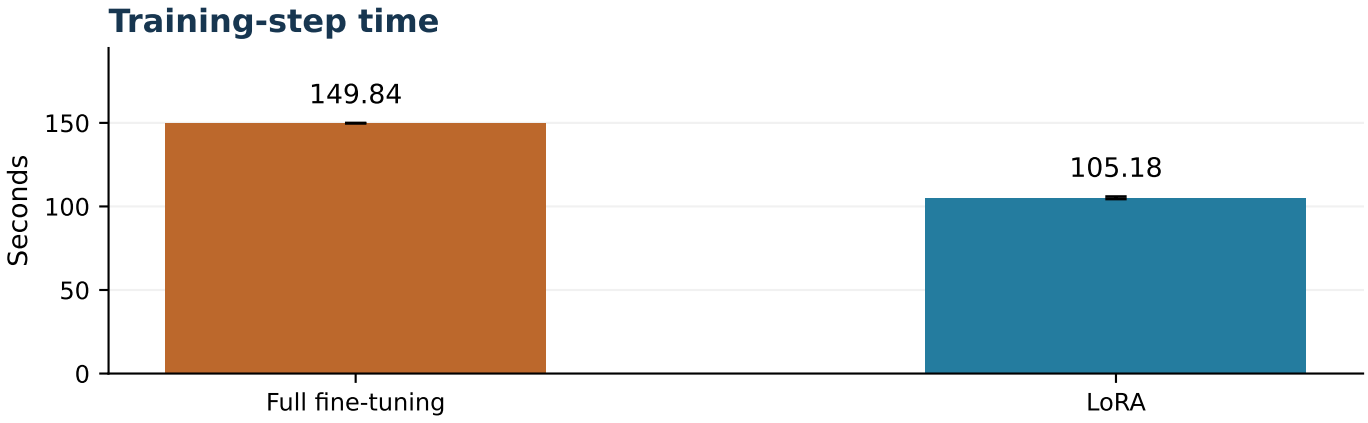
LoRA makes 0.3605% of its total parameters trainable. Every LoRA base-weight audit passed for all three seeds. Full fine-tuning changed 123,851,224 values in seed 42; LoRA changed all 589,824 adapter values. The chart uses a zero baseline; small adapter bars are labeled explicitly.

Training resources: what is saved?

NVIDIA A30 | FP32 | 1,200 steps | batch 8 | sequence length 128



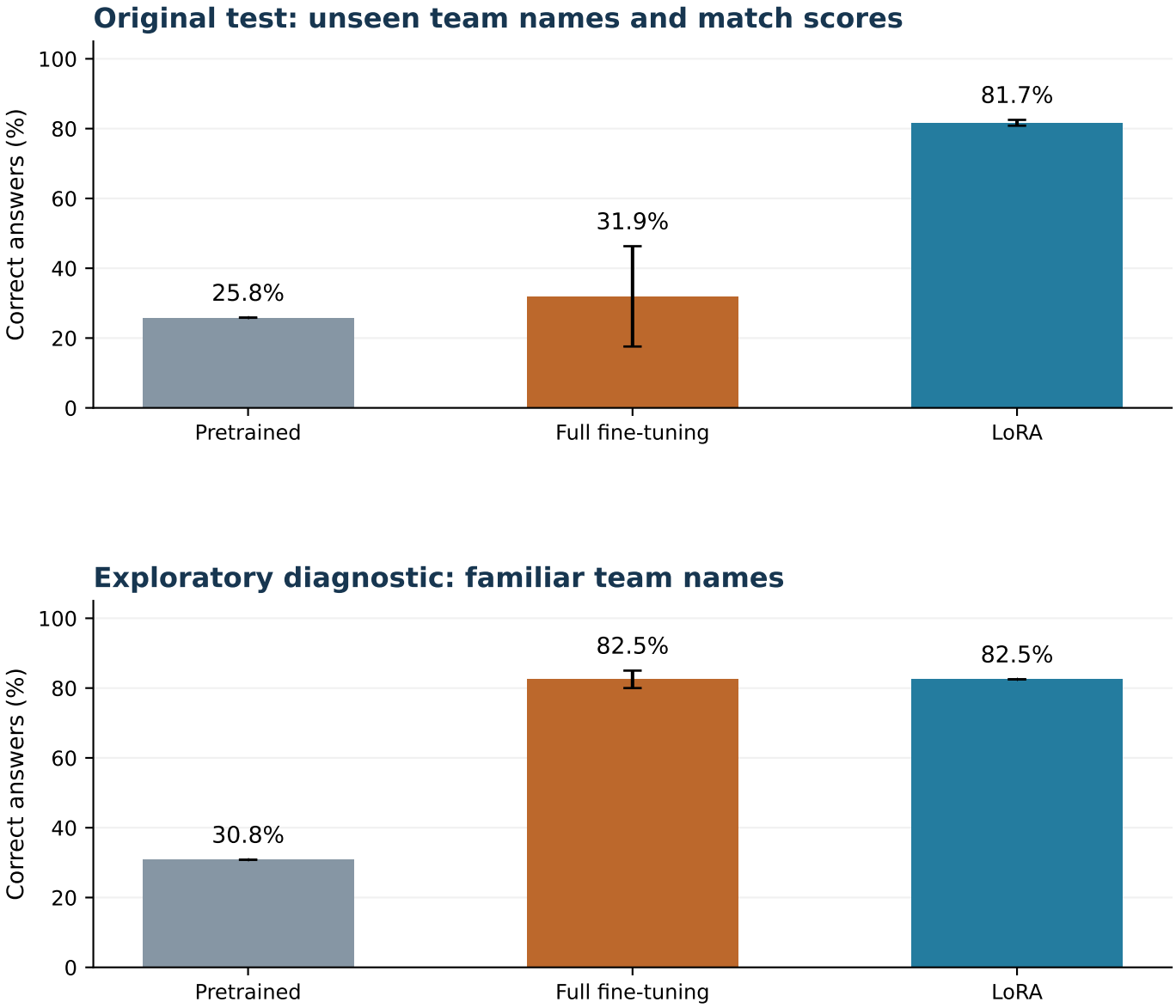




LoRA adds 2.25 MiB of FP32 weights, but reduces measured peak allocated training memory by 33.8% and training-step time by 29.8%. Runtime memory also includes gradients, optimizer states, activations, and temporary allocations. Time excludes transfers, validation, audits, checkpointing, and testing. Bars show three-seed means; error bars show sample standard deviation.

Quality alongside parameter efficiency

Correct primary answers | means and sample standard deviations over three seeds



Original test: 120 questions, with test team names and score combinations absent from training; none of its full reference answers occur in training. LoRA generalized better to unfamiliar names in this experiment. Full fine-tuning often substituted training-team names.

The familiar-name diagnostic was added after inspecting failures. Only names changed; scores and checkpoints stayed fixed. 9 of 120 reference answers overlap training. Its similar accuracy is encouraging but does not establish statistical equivalence.

These are synthetic, format-sensitive question-answering scores, not broad conversational quality. Both methods scored around 50% on comeback questions. Error bars are seed-to-seed variation, not confidence intervals.

Source: saved matched experiment reports | Conceptual diagrams labeled separately

LoRA versus full fine-tuning

MQP | Preliminary FP32 results | Prepared October 6, 2026

Purpose and comparison

Measure whether LoRA can learn a new task while training fewer parameters and using less GPU memory. We compare an unchanged pretrained reference, full fine-tuning (all original weights trainable), and LoRA fine-tuning (only added adapters trainable). Both trained methods receive exactly the same new data.

Experiment configuration

Pretrained GPT-2 in minGPT: 12 layers, 768 hidden units, 12 attention heads. Separate input and output embedding weights give 163,037,184 base parameters. Synthetic basketball data: 480 training matches / 1,440 questions; 40 validation matches / 120 questions; 40 test matches / 120 questions. Questions cover winners, halftime leaders, and comebacks.

Matched training protocol

NVIDIA A30; FP32; TF32 disabled. Batch size 8, padded sequence length 128, 1,200 steps, seeds 42/43/44. AdamW, zero weight decay, gradient clipping 1.0. LoRA rank 4, alpha 8, applied to attention and MLP projections. Starting weights, examples, sample order, and token budgets match between methods.

Model selection

Each method tried learning rates 0.001 and 0.0001 with seed 42. Validation loss selected full fine-tuning at 0.0001 and LoRA at 0.001; the chosen rates were then used for seeds 43 and 44. All selected checkpoints were at step 1,200. Test evaluation used identical prompts and greedy generation, capped at 40 new tokens.

Parameter counts

Measurement	Full fine-tuning	LoRA
Total parameters	163,037,184	163,627,008
Trainable parameters	163,037,184	589,824
Trainable fraction	100%	0.3605%
Changed scalar values (seed 42)	123,851,224	589,824
Frozen base values changed	Not applicable	0

How the frozen-weight claim was verified

Parameters were compared with cloned CPU snapshots of their initial values. Exact scalar comparisons distinguish values that actually changed from those merely allowed to change. All LoRA base-weight audits passed: zero original weight values changed. LoRA has 276.4 times fewer trainable parameters.

Source: saved matched-run reports, checkpoints, and familiar-name diagnostic.

Memory and training time

MQP | Preliminary FP32 results | Prepared October 6, 2026

Measured resource results

Measurement	Full fine-tuning	LoRA
FP32 parameter storage	621.94 MiB	624.19 MiB
Peak allocated GPU memory	3,747.0 MiB	2,481.6 MiB
Peak reserved GPU memory	4,060.0 MiB	2,758.0 MiB
Mean training-step time	149.8 seconds	105.2 seconds

Interpretation

LoRA reduced peak allocated training GPU memory by 33.8% and synchronized training-step time by 29.8%. Weight storage increased by 2.25 MiB because adapters are additional parameters. LoRA saves training resources while retaining the full pretrained model.

Storage is different from runtime memory

FP32 parameter storage equals parameter count multiplied by 4 bytes (32 bits). Peak allocated training memory also reflects live gradients, optimizer states, activations, and other PyTorch allocations. Reserved memory includes allocator-held space; it must not be added to allocated memory. These are PyTorch measurements, not total device memory including every driver allocation.

Timing scope

Training-step time is averaged over three seeds and covers the 1,200 synchronized training steps. It excludes data transfer, validation, parameter audits, checkpoint saving, and test generation. These results compare a fixed training budget; they do not establish time required to reach a particular quality target.

Baseline controls

FP16/BF16, FlashAttention, and KV caching were not introduced into this comparison. Keeping the experiment in FP32 isolates LoRA as requested. Combined-technique experiments and broader latency/throughput scaling remain future work.

Source: saved matched-run reports, checkpoints, and familiar-name diagnostic.

Quality, examples, and conclusions

MQP | Preliminary FP32 results | Prepared October 6, 2026

Original held-out test

Metric	Pretrained	Full fine-tuning	LoRA
Correct primary answer	25.8%	31.9 +/- 14.4%	81.7 +/- 0.8%
Answer and score fields	0.0%	31.9 +/- 14.4%	80.8 +/- 1.4%
Both score-swap answers correct	0.0%	6.4 +/- 7.7%	65.8 +/- 0.8%

What this test measures

Test team names and match-score combinations were absent from training, and none of the full reference answers appeared in training. Score-swapped questions test whether answers follow changed inputs. Reported +/- values are sample standard deviations across three training seeds, not confidence intervals. The benchmark was previously disclosed, not newly blinded.

Actual held-out example (seed 42)

Prompt: Bulls vs Sharks; halftime 51-35; final 96-69. Who won? Full fine-tuning: "Rockets won 96-69." LoRA: "Bulls won 96-69." The pretrained model generated contradictory winner statements. Full fine-tuning often substituted names seen during training.

Exploratory familiar-name diagnostic

After observing those failures, we replaced only test team names with familiar names, keeping scores and checkpoints unchanged. Primary-answer accuracy: pretrained 30.8%; full fine-tuning 82.5 +/- 2.5%; LoRA 82.5 +/- 0.0%. Both trained models correctly answered "Bears won 96-69" for the corresponding Bears-Hawks example. This diagnostic involved no retraining, but 9 of 120 reference answers overlap training.

Limits on the quality claim

This is synthetic, template-based basketball question answering, not broad sports conversation. Scoring uses format-sensitive answer and score-field checks. Questions share matches, and there are only three training seeds. Both methods achieved roughly 50% on comeback questions. The familiar-name diagnostic is exploratory and does not prove statistical equivalence; the unseen-name result does not establish that LoRA universally outperforms full fine-tuning.

Finding to present

Both methods learned the task with familiar team names. LoRA made only 0.3605% of its parameters trainable, preserved every base weight, and used less measured GPU memory and training-step time. It also generalized better to unfamiliar team names in this experiment. These are preliminary results supporting parameter efficiency, with clear remaining quality limitations.

Source: saved matched-run reports, checkpoints, and familiar-name diagnostic.